

PID Controllers – Worked Problems

Practice: tune and compare PID controllers.

Ogata, *Modern Control Engineering*, Ch. 8 (end-of-chapter style).

Three problems: Ziegler-Nichols tuning, ZN vs. hand-tuning, and a PI-vs-PID steady-state/overshoot tradeoff. Step through with **Ctrl+Enter**, or render a report with **publish**.

Contents

- Problem 1: Ziegler-Nichols Method 2 Tuning
- Problem 1 – Before vs. After
- Problem 2: Compare ZN Tuning to a Hand-Tuned PID
- Problem 3: PI-Only vs. Full PID Steady-State and Transient Tradeoff
- Try it yourself

Problem 1: Ziegler-Nichols Method 2 Tuning

For $G(s) = \frac{1}{s(s+1)(s+2)}$, find the ultimate gain K_{cr} and ultimate period P_{cr} , then compute the ZN Method 2 PID parameters.

```
G1 = tf(1,conv([1 0],conv([1 1],[1 2])));

% Characteristic equation: s(s+1)(s+2)+K = s^3+3s^2+2s+K = 0
% Routh s^1 row: (3*2-K)/3 = 0 -> K = 6
Kcr1 = 6;
omega_cr1 = sqrt(2); % from auxiliary eqn 3s^2+K=0 -> s^2=-K/3=-2
Pcr1 = 2*pi/omega_cr1;
fprintf('Problem 1: Kcr = %.4f, Pcr = %.4f s\n', Kcr1, Pcr1)

Kp1 = 0.6*Kcr1; Ti1 = 0.5*Pcr1; Td1 = 0.125*Pcr1;
fprintf('Problem 1 PID: Kp=%.4f, Ti=%.4f, Td=%.4f\n', Kp1, Ti1, Td1)

Gc1 = tf(Kp1*[Td1 1 1/Ti1],[1 0]);
T1 = feedback(Gc1*G1,1);

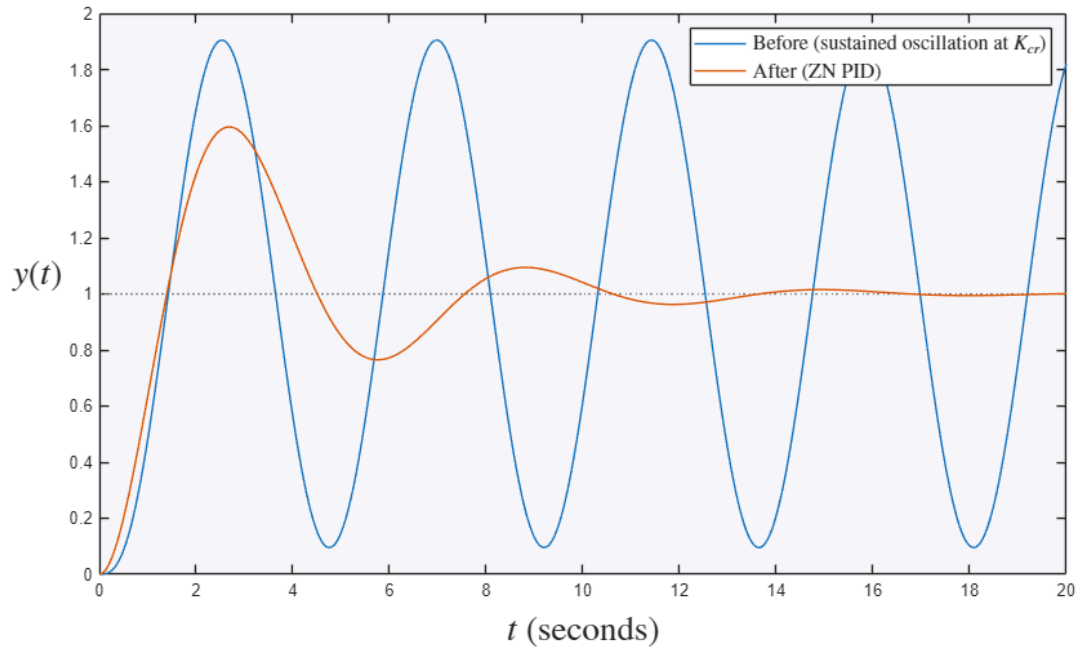
Problem 1: Kcr = 6.0000, Pcr = 4.4429 s
Problem 1 PID: Kp=3.6000, Ti=2.2214, Td=0.5554
```

Problem 1 – Before vs. After

”Before” is the diagnostic the tuning rule starts from: at the ultimate gain $K=K_{cr}$ the closed loop oscillates without decay. ”After” applies the ZN Method 2 PID, which damps the response into a usable step.

```
T_marg = feedback(Kcr1*G1,1);
figure
step(T_marg,0:0.01:20)
hold on
step(T1,0:0.01:20)
hold off
legend('Before (sustained oscillation at $K_{cr}$)', 'After (ZN PID)', 'Interpreter', 'latex', 'FontSize', 12)
title('Problem 1: Before vs. After ZN Method 2 Tuning', 'Interpreter', 'latex', 'FontSize', 16)
ylabel('$y(t)$', 'Interpreter', 'latex', 'FontSize', 20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$', 'Interpreter', 'latex', 'FontSize', 20)
```

Problem 1: Before vs. After ZN Method 2 Tuning



Problem 2: Compare ZN Tuning to a Hand-Tuned PID

Reduce overshoot from Problem 1 by hand-tuning down K_p and T_d while preserving zero steady-state error (T_i retained for integral action).

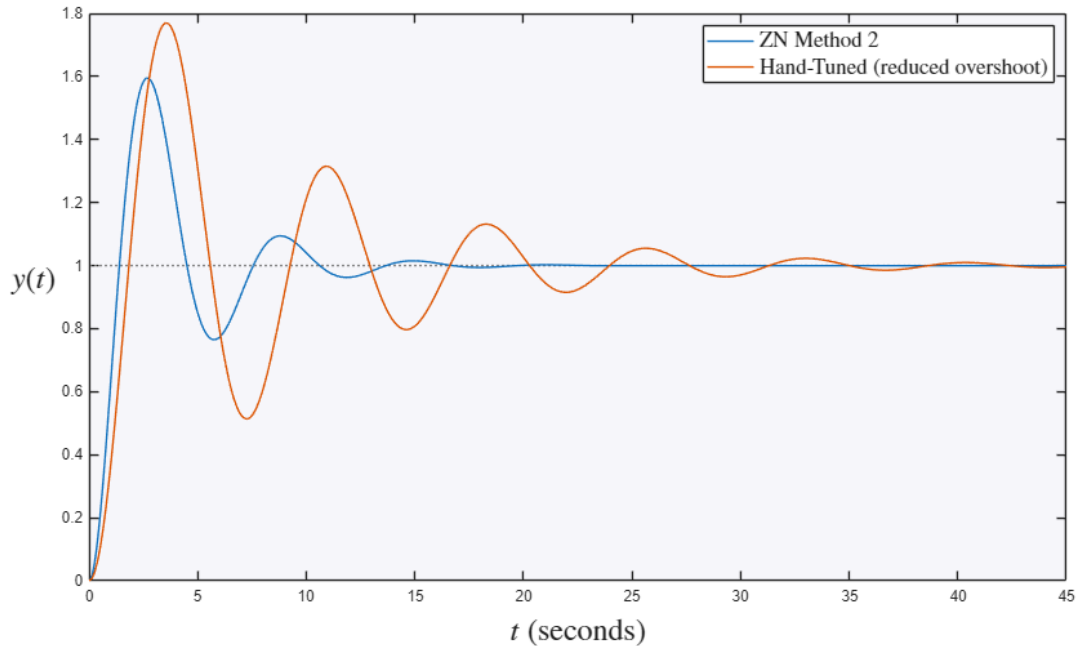
```
Kp2 = 0.4*Kcr1; Ti2 = Ti1; Td2 = 0.08*Pcr1;
Gc2 = tf(Kp2*[Td2 1 1/Ti2],[1 0]);
T2 = feedback(Gc2*G1,1);

info1 = stepinfo(T1);
info2 = stepinfo(T2);
fprintf('Problem 2: ZN overshoot = %.2f%%, hand-tuned overshoot = %.2f%%\n', ...
        info1.Overshoot, info2.Overshoot)
fprintf('Problem 2: ZN settling = %.4f s, hand-tuned settling = %.4f s\n', ...
        info1.SettlingTime, info2.SettlingTime)

figure
hold on
step(T1)
step(T2)
hold off
legend('ZN Method 2','Hand-Tuned (reduced overshoot)','Interpreter','latex','FontSize',14)
title('Problem 2: ZN vs. Hand-Tuned PID','Interpreter','latex','FontSize',20)
ylabel('$y(t)$','Interpreter','latex','FontSize',20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$','Interpreter','latex','FontSize',20)
```

```
Problem 2: ZN overshoot = 59.48%, hand-tuned overshoot = 76.99%
Problem 2: ZN settling = 12.9177 s, hand-tuned settling = 33.5931 s
```

Problem 2: ZN vs. Hand-Tuned PID



Problem 3: PI-Only vs. Full PID Steady-State and Transient Tradeoff

For $G(s) = \frac{2}{(s+1)(s+3)}$, design (a) a PI controller for zero steady-state step error and (b) a full PID adding derivative action to reduce the resulting overshoot, then compare.

```
G3 = tf(2,conv([1 1],[1 3]));

Kp3 = 4; Ti3 = 1.5;
Gc3_PI = tf(Kp3*[1 1/Ti3],[1 0]);
T3_PI = feedback(Gc3_PI*G3,1);

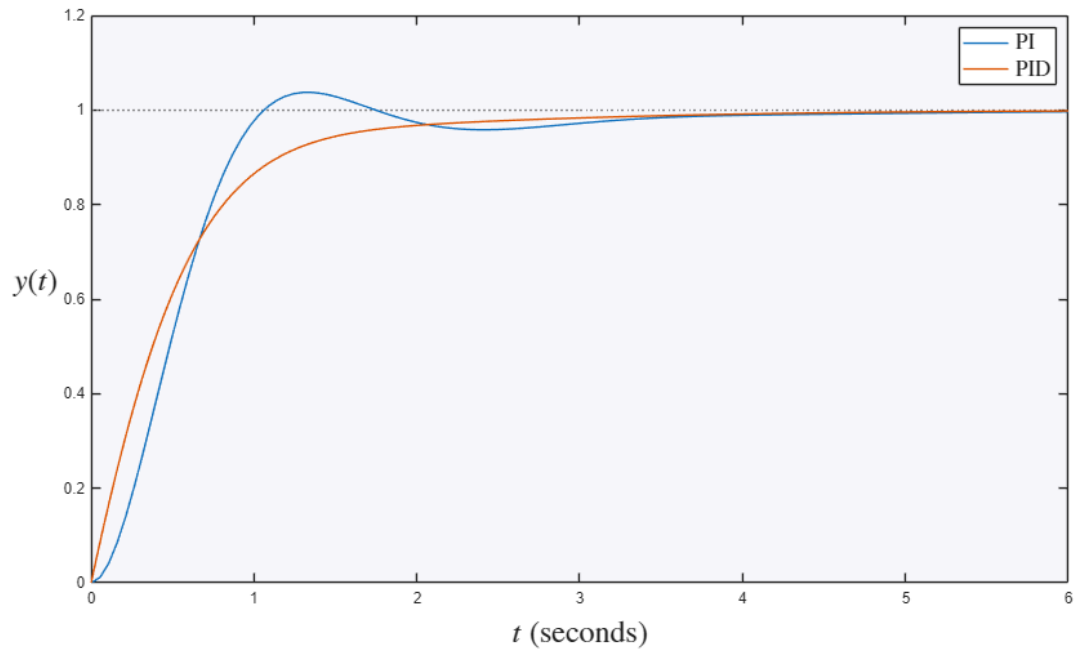
Td3 = 0.2;
Gc3_PID = tf(Kp3*[Td3 1 1/Ti3],[1 0]);
T3_PID = feedback(Gc3_PID*G3,1);

info3_PI = stepinfo(T3_PI);
info3_PID = stepinfo(T3_PID);
fprintf('Problem 3: PI overshoot=%.2f%%, PID overshoot=%.2f%%\n', ...
        info3_PI.Overshoot, info3_PID.Overshoot)

figure
hold on
step(T3_PI)
step(T3_PID)
hold off
legend('PI','PID','Interpreter','latex','FontSize',14)
title('Problem 3: PI vs. PID Overshoot Reduction','Interpreter','latex','FontSize',20)
ylabel('$y(t)$','Interpreter','latex','FontSize',20)
set(get(gca,'YLabel'),'Rotation',0)
xlabel('$t$','Interpreter','latex','FontSize',20)
```

```
Problem 3: PI overshoot=3.72%, PID overshoot=0.00%
```

Problem 3: PI vs. PID Overshoot Reduction



Try it yourself

- In Problem 2, keep lowering K_{p2} and T_{d2} until the overshoot is under 5%, then compare settling time with the ZN design.
- In Problem 3, drop the derivative term and confirm the overshoot climbs back – derivative action is doing the damping.